Advanced Deep Learning

Paul Swoboda

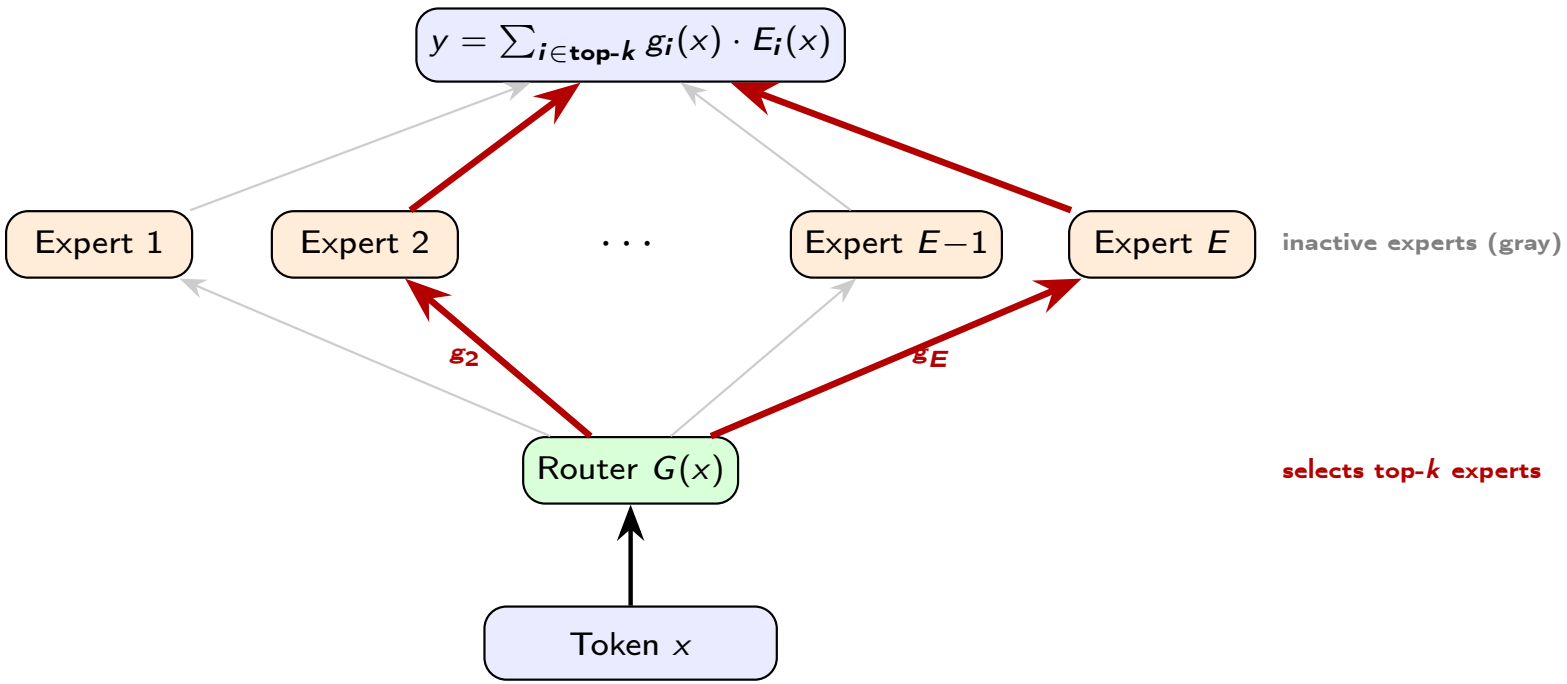
HHU Düsseldorf

Summer Semester 2026

Table of Contents

Table of Contents

MoE: High-Level Idea



- Replace the dense FFN in each Transformer block with a **sparse MoE layer**
- Maintain E expert sub-networks, each a standard FFN
- A **router** (gating network) selects the top-k experts per token
- Output: weighted combination $y = \sum_{i \in \text{top-}k} g_i(x) \cdot E_i(x)$

Why Mixture of Experts?

Dense models: every parameter is used for every input token.

- Scaling parameters $\Rightarrow$ scaling FLOPs proportionally
- Training cost grows with model size
- Inference cost grows with model size

Key insight: Not all tokens need the same computation. Can we *decouple* parameter count from compute cost?

Analogy: Like consulting a team of specialists — you don't ask every expert for every question, only the relevant ones.

Mixture of Experts (MoE)

- Maintain *many* expert sub-networks
- For each token, activate only a *few* experts
- $\Rightarrow$ Large model capacity with sparse computation

Result: Models with N total parameters but only $\sim k/E \cdot N$ active parameters per token (k active out of E experts).

In practice:

- Experts tend to *specialize* on different token types, languages, or domains
- MoE models match dense model quality at 2–4$\times$ less training compute
- Inference latency similar to a small dense model (only k experts run)
- Most successful modern LLMs use MoE: Mixtral, DeepSeek-V3, Grok

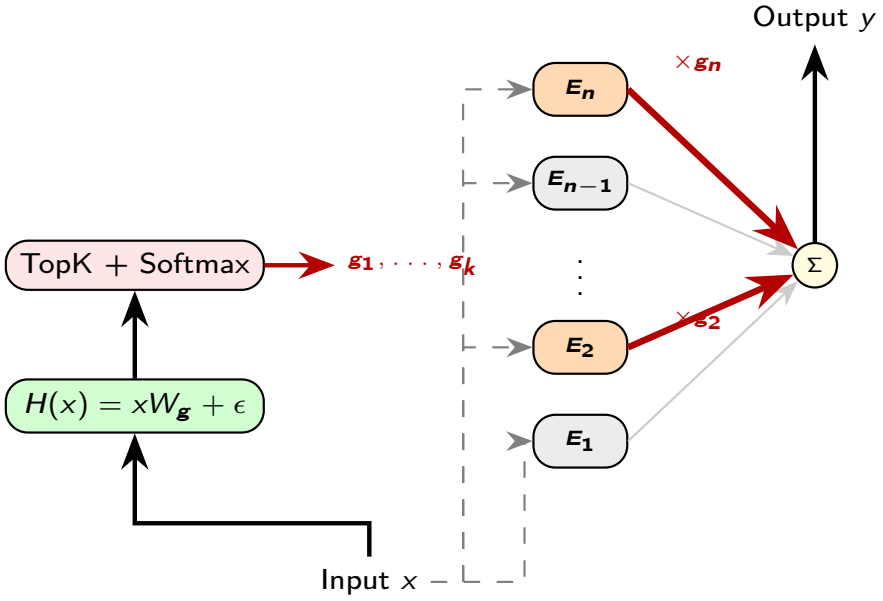
Challenges of MoE

1. **Top-k is not differentiable**
   - The discrete selection of which k experts to activate is a hard, non-differentiable choice
   - Gradients cannot flow through $\mathrm{argtopk}$
   - **Workaround**: Multiply each expert's output by its continuous gating weight $g_i(x)$. Gradients flow through g_i via softmax over the selected experts
   - Router weights W_g are trained by the gating weights, not the selection decision
2. **Load balancing**
   - Router tends to collapse: sends most tokens to a few "favorite" experts
   - Favored experts improve faster $\Rightarrow$ attract even more tokens (self-reinforcing)
   - Requires auxiliary losses or other mechanisms
3. **Communication overhead**
   - Experts distributed across GPUs
   - Tokens must be dispatched to the right device via **all-to-all** communication
   - Poor load balance $\Rightarrow$ stragglers and wasted compute
4. **Memory vs. compute trade-off**
   - All expert weights must reside in memory, even though most are idle per token
   - Total parameter count $\gg$ active parameters
   - Inference requires careful memory management (e.g. expert offloading)
5. **Training instability**
   - Router learning is noisy early in training
   - Requires careful initialization and precision (e.g. router in `float32`)

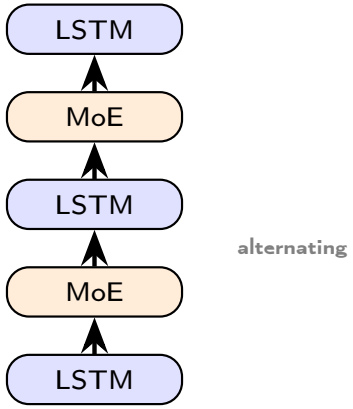
Table of Contents

Sparsely-Gated MoE: Architecture

shazeer2017moe: up to **131,072 experts**, top-k gating (k=2).



Placement in network:



- MoE layers inserted *between* stacked LSTM layers
- Each MoE receives the preceding LSTM's output
- LSTMs: sequential modeling
- MoE: adds capacity without proportional compute
- Up to 5 LSTM + MoE pairs

The Sparsely-Gated Mixture-of-Experts Layer

Gating network for input x:

$$G(x) = \mathrm{softmax}\Big(\mathrm{TopK}\big(H(x),\ k\big)\Big)$$

$$\mathrm{TopK}(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \\ -\infty & \text{otherwise} \end{cases}$$

Noisy gating (training only, disabled at inference):

$$H(x)_i = (x \cdot W_g)_i + \underbrace{\mathcal{N}(0,1) \cdot \mathrm{Softplus}\big((x \cdot W_{\text{noise}})_i\big)}_{\text{learned, input-dependent noise}}$$

- W_{noise} is a second learned weight matrix controlling per-expert noise magnitude
- Noise encourages *exploration*: prevents the router from always picking the same experts
- At inference, noise is turned off $\Rightarrow$ deterministic routing

MoE output: $y = \sum_{i=1}^{E} G(x)_i \cdot E_i(x)$ (only k terms non-zero)

Load Balancing

As discussed earlier, the router tends to collapse without intervention. Noisy gating helps but is not sufficient alone.

Auxiliary load-balancing loss (added to main loss):

For a batch of T tokens across E experts:

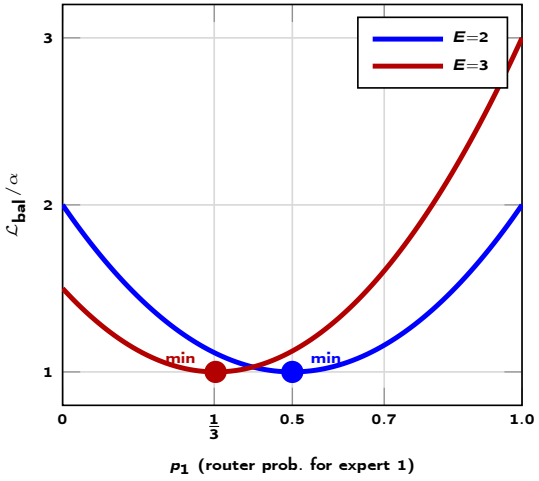
- $f_i = \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}[\text{token } t \to \text{expert } i]$ (hard, non-differentiable)
- $p_i = \frac{1}{T} \sum_{t=1}^{T} \mathrm{softmax}(H(x_t))_i$ (soft, differentiable)

$$\mathcal{L}_{\text{bal}} = \alpha \cdot E \cdot \sum_{i=1}^{E} f_i \cdot p_i$$

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \mathcal{L}_{\text{bal}}$$

- Minimized when $f_i = p_i = 1/E$ (uniform)
- Gradients flow through p_i, pushing router probabilities toward uniformity
- α small (e.g. 10^{-2}) to not dominate

Illustration (E=3, assuming $f_i \approx p_i$):



Loss as a function of how much probability is concentrated on one expert (others split equally). Minimum at uniform $p_i = 1/E$.

Trade-off: Too small $\alpha \Rightarrow$ imbalance; too large $\Rightarrow$ hurts quality by forcing uniformity.

Results: Scaling with Sparse MoE

shazeer2017moe demonstrated on language modeling and machine translation:

- **1000× more parameters** than a dense model at similar computational cost
- Achieved state-of-the-art results on language modeling (1 Billion Word Benchmark) and machine translation
- A model with **137B parameters** (68B active per token with $k = 2$ of 2048 experts) matched or exceeded much larger dense models

Key Takeaway

Sparse MoE enables *conditional computation*: the model can store vastly more knowledge in its parameters without proportionally increasing the compute per example.

Challenges identified:

- Load balancing is critical and non-trivial
- Communication overhead when experts are distributed across devices
- Batch size must be large enough so each expert sees sufficient tokens

Table of Contents

Switch Transformer: Simplifying MoE Routing

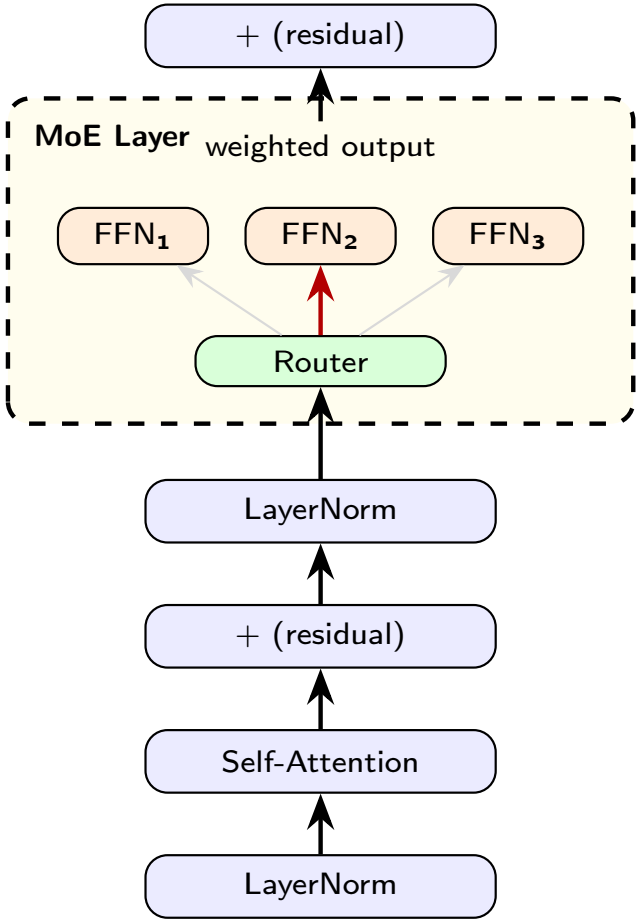
fedus2022switch simplified the MoE approach with two key changes:

1. **Top-1 routing** ("Switch" routing):
   - Each token goes to *exactly one* expert
   - Simpler than top-k: no need to combine outputs
   - Reduces communication and computation

$$i^* = \arg\max_i (x \cdot W_r)_i$$

$$y = G(x)_{i^*} \cdot E_{i^*}(x)$$

2. **Expert capacity factor**:
   - Each expert processes at most C tokens per batch
   - $C = \frac{k \cdot T}{E} \cdot c$, with capacity factor $c > 1$ (e.g. $c = 1.25$)
   - Overflow tokens are passed through residual connection ("dropped")



Switch Transformer: Results

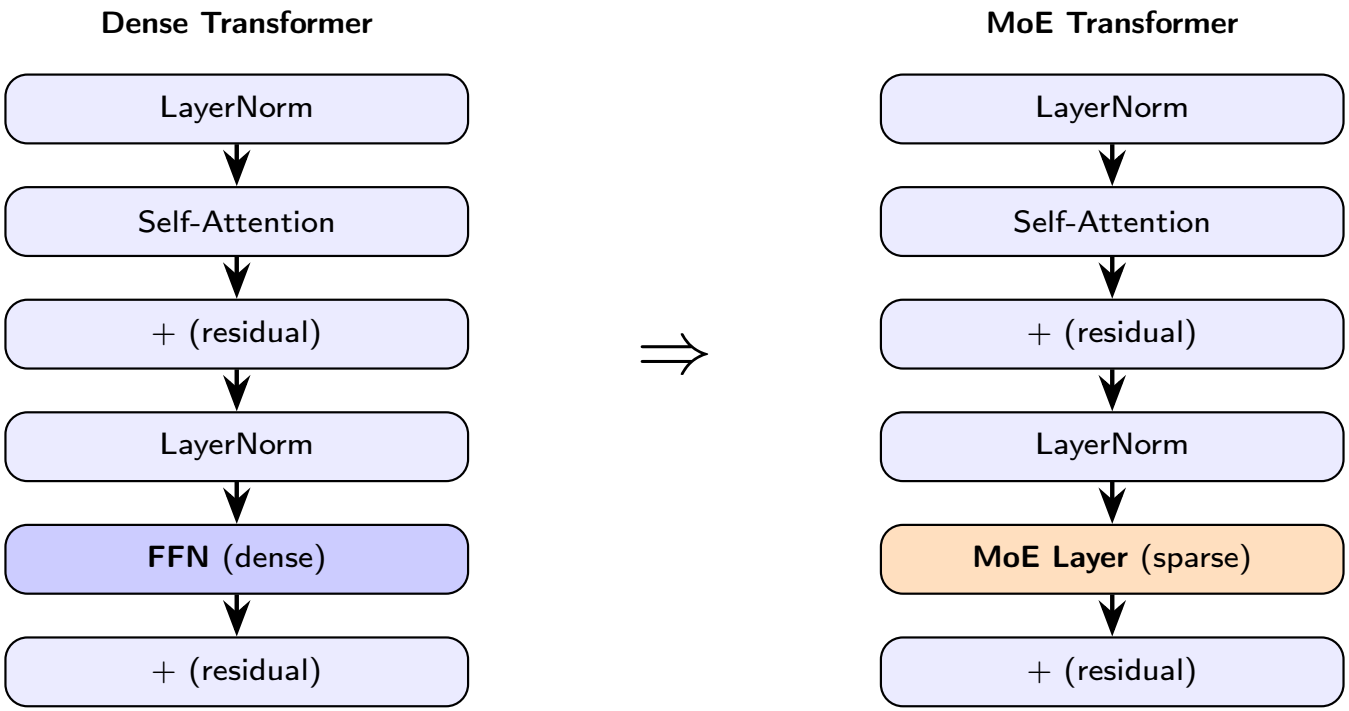
- Scaled up to **1.6 trillion parameters** (Switch-C) with 2048 experts
- At matched FLOPs, Switch Transformers achieve **4–7**$\times$ **speedup** over dense T5 baselines
- Training instability at large scale mitigated by:
  - Selective use of `bfloat16` (router in `float32`)
  - Smaller parameter initialization
  - Higher expert dropout during fine-tuning

Switch Transformer Design Principles

1. Simplify routing: top-1 is enough
2. Use large but efficient expert count
3. Keep auxiliary loss for load balancing
4. Careful numerical precision for stability

Table of Contents

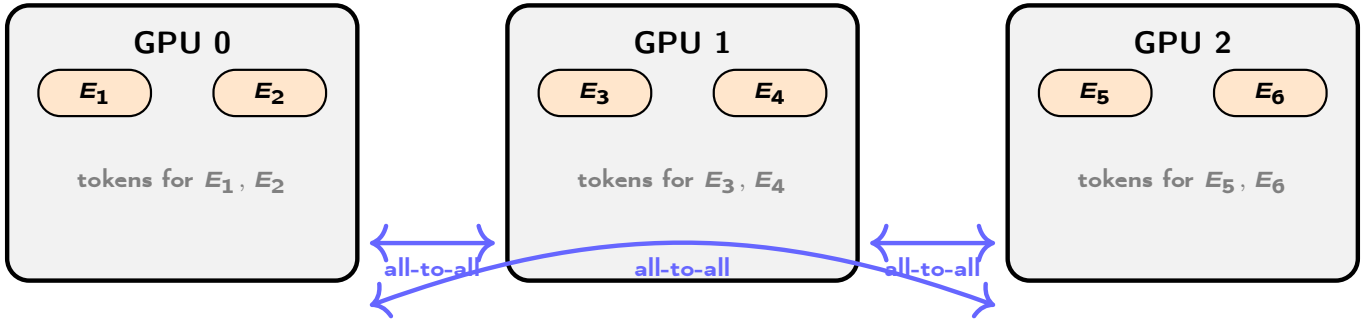
Where Does MoE Fit in a Transformer?



- The **FFN sub-layer** is replaced by the MoE layer (every layer or every n-th layer)
- Attention layers remain **dense** (shared across all tokens)
- Common pattern: alternate dense FFN and MoE layers

Expert Parallelism

When experts are distributed across devices, MoE introduces a new parallelism dimension:



- **Dispatch**: **All-to-all** communication sends each token to the GPU hosting its assigned expert
- **Combine**: After expert computation, **all-to-all** sends results back
- **Cost**: Communication volume depends on load balance — poor balance ⇒ congestion

Table of Contents

DeepSeek-MoE: Fine-Grained Expert Specialization

dai2024deepseekmoe proposed two innovations to improve expert specialization:

1. **Fine-grained expert segmentation**:
   - Instead of E large experts, split into mE smaller experts (each $1/m$ the size)
   - Activate mk experts (same total compute)
   - Allows more flexible, fine-grained combinations of knowledge

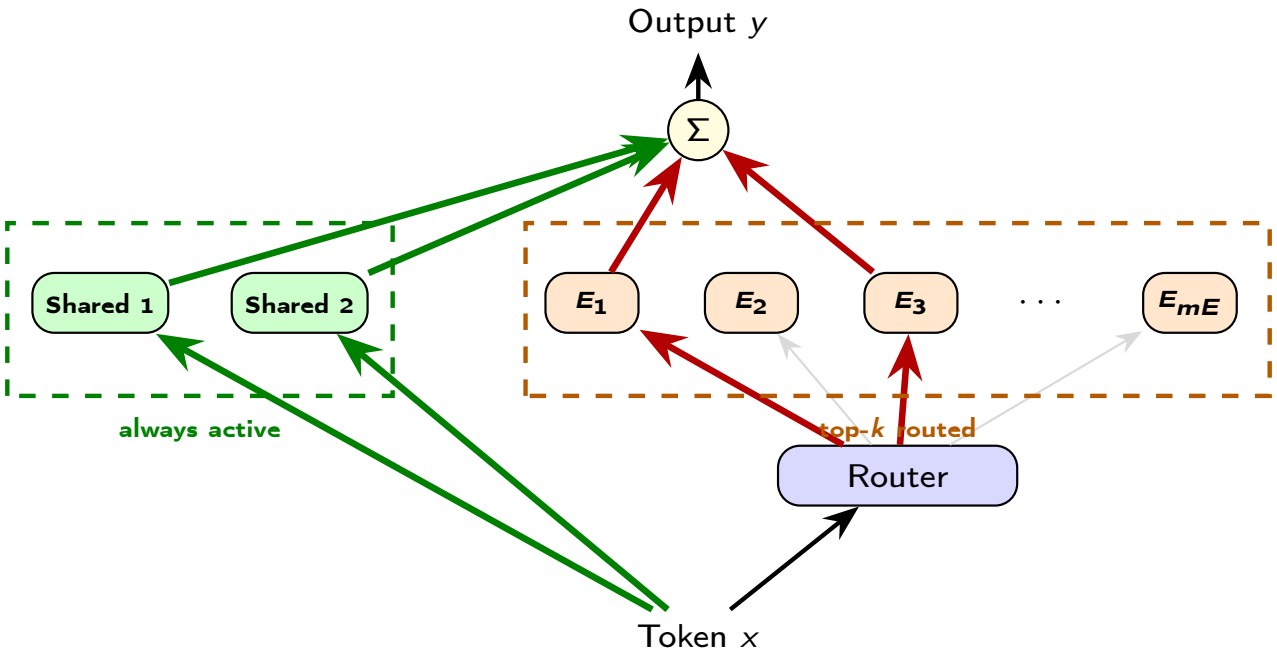
Example: Instead of 16 experts with top-2, use 64 experts with top-8 (same compute, more combinatorial flexibility).

2. **Shared experts**:
   - Dedicate K_s experts as **shared** — always active for every token
   - Remaining experts are **routed** as usual
   - Shared experts capture common knowledge; routed experts specialize

Output

$$y = \sum_{i=1}^{K_s} E_i^{(s)}(x) + \sum_{i \in \text{top-}k} g_i \cdot E_i^{(r)}(x)$$

DeepSeek-MoE: Architecture



- Shared experts reduce *redundancy* among routed experts
- Fine-grained segmentation provides $\binom{mE}{mk}$ possible expert combinations vs. $\binom{E}{k}$ with standard MoE

DeepSeek-MoE: Results

DeepSeek-MoE 16B (2B active parameters, 64 routed experts + 2 shared):

- Comparable performance to LLaMA-2 7B (3.5× more active params)
- Comparable performance to DeepSeek 7B dense model
- ≈40% of the compute of a dense model with equivalent quality

Scaling validation at 145B parameters:

- DeepSeek-MoE 145B approaches the performance of DeepSeek 67B dense
- Only 28.5B active parameters (vs. 67B)
- Confirms that fine-grained + shared experts scale well

Design Recipe

Fine-grained segmentation ($m \approx 4$) + shared experts ($K_s = 2$) consistently outperforms both standard top-k MoE and dense models at matched compute.

Table of Contents

DeepSeek-V3: Auxiliary-Loss-Free Load Balancing

deepseekai2024deepseekv3 identified a key problem with auxiliary balancing losses:

Problem

The auxiliary loss $\mathcal{L}_{\text{bal}}$ conflicts with the language modeling objective. Tuning α is difficult: too small $\Rightarrow$ imbalance, too large $\Rightarrow$ hurts model quality.

Solution: Bias-based load balancing (no auxiliary loss):

- Add a learnable **bias** term b_i to each expert's gating score for routing:

$$i^* = \arg\max_i \left(s_i(x) + b_i\right)$$

- The bias is **not** used in the gating weight (only for routing decisions):

$$g_i(x) = \frac{s_i(x)}{\sum_{j \in \text{top-}k} s_j(x)}$$

- b_i is adjusted dynamically: if expert i is overloaded, decrease b_i; if underloaded, increase b_i

$\Rightarrow$ Load balancing is *decoupled* from the training loss.

DeepSeek-V3: Architecture Overview

DeepSeek-V3: 671B total parameters, 37B active per token.

MoE configuration:

- 61 Transformer layers
- First 3 layers: dense FFN
- Layers 4–61: MoE layers
- 256 routed experts + 1 shared expert per MoE layer
- Top-8 routing (out of 256)
- Fine-grained segmentation following DeepSeek-MoE

Additional innovations:

- Multi-head Latent Attention (MLA) for KV cache compression
- Multi-token prediction as auxiliary training objective
- FP8 mixed-precision training

Training efficiency:

- Trained on 14.8T tokens
- Training cost: **2.788M H800 GPU hours**
- ≈$5.5M (remarkably economical)
- Comparable to GPT-4 / LLaMA-3 405B quality

Why so efficient?

- MoE: only 37B of 671B params active
- FP8 training: ∼2× throughput vs. BF16
- Efficient expert parallelism with minimal communication

Comparison: Dense vs. MoE Approaches

Model	Total Params	Active Params	MoE Design
LLaMA-3 405B	405B	405B	Dense
Mixtral 8×7B	47B	13B	Top-2 of 8
Mixtral 8×22B	141B	39B	Top-2 of 8
DeepSeek-V2	236B	21B	Top-6 of 160 + shared
DeepSeek-V3	671B	37B	Top-8 of 256 + shared

Trends in MoE design:

1. **More, smaller experts**: from 8 large → 256 fine-grained
2. **Higher top-k**: from top-1/2 → top-6/8 with smaller experts
3. **Shared experts**: reduce redundancy, capture common knowledge
4. **Better load balancing**: from auxiliary losses → bias-based methods

Table of Contents

MoE: Summary and Open Challenges

Key ideas:

- MoE *decouples* model capacity (total params) from compute cost (active params)
- Router selects a sparse subset of experts per token
- Load balancing is essential for training stability and efficiency

Evolution:

- **Shazeer (2017)**: Sparsely-gated MoE layer, noisy top-k gating, auxiliary loss
- **Switch (2022)**: Simplified top-1 routing, scaled to 1.6T params
- **DeepSeek-MoE (2024)**: Fine-grained experts + shared experts
- **DeepSeek-V3 (2024)**: Auxiliary-loss-free balancing, 671B params at low cost

Open challenges:

- **Inference**: All expert weights must be in memory, even though most are idle
- **Expert collapse**: Some experts may remain underutilized
- **Communication**: Expert parallelism requires all-to-all communication
- **Distillation**: Can we compress MoE back into smaller dense models?

Table of Contents

References I

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some
data that should have been added to the final page this extra page has been adde
If you rerun the document (without altering it) this surplus page will go away, bec
knows how many pages to expect for this document.